

[33] Neo4j: Vector index and search 총정리

개발사: Neo4j

형태: 그래프 데이터베이스의 벡터 검색 통합 기능

웹사이트: <https://neo4j.com/>

공개/업데이트: 2024

요약

Neo4j 는 그래프 데이터베이스로 잘 알려져 있으며, 최근 벡터 인덱싱과 검색 기능을 핵심 플랫폼에 통합했다. Neo4j 의 벡터 검색은 단순한 임베딩 저장에 그치지 않고, 그래프의 노드와 관계와 함께 벡터 데이터를 저장하고 검색하는 통합된 접근이다. 이를 통해 의미적 유사도(벡터 기반)와 구조적 유사도(그래프 관계 기반)를 함께 고려하는 강력한 검색이 가능해진다. 지식 그래프, 추천 시스템, 의미 검색 등 다양한 응용에서 그래프 구조와 의미적 정보를 동시에 활용할 수 있다.

상세분석

33.1 주요 문제점과 설계 목표

그래프 데이터베이스 환경에서 벡터 검색의 필요성:

- **의미적 검색의 부재:** 기존 그래프 DB 는 명시적 관계와 속성 기반 검색만 지원, 의미적 유사도 검색 불가
- **구조와 의미의 분리:** 관계 정보와 벡터 정보를 별도 시스템에서 관리해야 했던 문제
- **지식 그래프의 완성도:** 엔티티 간 벡터 기반 유사도를 고려한 검색 필요
- **추천 시스템의 통합:** 사용자-아이템 관계와 벡터 유사도를 함께 고려하는 하이브리드 추천
- **오버헤드:** 그래프와 벡터 DB 를 동시에 운영하는 복잡성과 데이터 동기화

Neo4j 의 해결책:

- 벡터를 노드의 속성으로 저장
- HNSW 기반 벡터 인덱싱
- 그래프 탐색과 벡터 검색의 통합 쿼리
- 관계 기반 검색에 벡터 필터 추가 가능

33.2 핵심 아키텍처

데이터 모델

그래프 + 벡터 통합:

노드(Node)

- ID: 고유 식별자
- 레이블(Label): 노드 타입
- 속성(Properties):
 - 일반 속성 (text, number, date 등)
 - 벡터 속성 (embedding)

관계(Relationship)

- 타입(Type): 관계 종류
- 속성(Properties): 관계의 메타데이터

예시 스키마:

```
// 문서 노드에 벡터 저장
CREATE (doc:Document {
  id: 'doc123',
  title: 'AI in Healthcare',
  content: '...',
  embedding: [0.1, 0.2, ..., 0.n] // 1536 차원 벡터
})

// 사용자 노드와의 관계
CREATE (user:User {name: 'Alice'})
CREATE (user)-[:INTERESTED_IN]->(doc)
```

벡터 인덱싱

HNSW 벡터 인덱스:

```
// 벡터 인덱스 생성
CREATE VECTOR INDEX doc_embedding_index
IF NOT EXISTS
FOR (n:Document)
ON n.embedding
OPTIONS {
  indexConfig: {
    `vector.dimensions`: 1536,
    `vector.similarity_function`: 'cosine',
    `vector.m`: 16,
    `vector.efConstruction`: 128,
    `vector.ef`: 40
  }
}
```

파라미터:

- **dimensions**: 벡터 차원 수
- **similarity_function**: 거리 메트릭 (cosine, euclidean, dot_product)
- **m**: HNSW 그래프 구조의 파라미터 (연결 수)
- **efConstruction**: 인덱스 생성 시 탐색 범위
- **ef**: 쿼리 시 탐색 범위

33.3 주요 기능

1. 순수 벡터 검색

유사 벡터 찾기:

```
CALL db.index.vector.queryNodes('doc_embedding_index', 10, [0.1, 0.2, ..., 0.n])
YIELD node, score
RETURN node.title, score
ORDER BY score DESC
```

결과:

- **node**: 매칭된 노드
- **score**: 유사도 점수 (거리 메트릭에 따라 정의)

2. 필터링을 포함한 벡터 검색

벡터 검색 + 노드 속성 필터:

```
CALL db.index.vector.queryNodes('doc_embedding_index', 100, query_vector)
YIELD node, score
WHERE node.category = 'healthcare'
  AND node.published_date > datetime('2024-01-01')
RETURN node.title, score
ORDER BY score DESC
LIMIT 10
```

3. 그래프 탐색과 벡터 검색의 결합

하이브리드 검색:

```
// 벡터 검색으로 초기 후보 찾기
CALL db.index.vector.queryNodes('doc_embedding_index', 50, query_vector)
YIELD node as similarDoc, score

// 그래프 탐색: 유사 문서와 관련된 다른 문서 찾기
MATCH (user:User {id: 'user123'})-[:VIEWED]->(doc:Document)
WHERE doc IN similarDoc
MATCH (doc)-[:RELATED_TO]->(relatedDoc:Document)

// 최종 점수: 벡터 유사도 + 그래프 관계 가중치
WITH relatedDoc, score,
  (score + 0.5) as hybridScore // 관계에 보너스 점수
ORDER BY hybridScore DESC
RETURN relatedDoc.title, hybridScore
```

4. 추천 시스템

협업 필터링 + 벡터 기반:

```
// 사용자와 유사한 사용자 찾기 (임베딩 기반)
CALL db.index.vector.queryNodes('user_embedding_index', 10, user_vector)
YIELD node as similarUser

// 유사 사용자가 좋아한 아이템 찾기 (그래프 관계 기반)
MATCH (similarUser)-[:RATED {score: >=4}]->(item:Item)
WHERE NOT (originalUser)-[:RATED]->(item)

// 아이템과 유사한 다른 아이템 찾기 (벡터 기반)
WITH item
CALL db.index.vector.queryNodes('item_embedding_index', 5, item.embedding)
YIELD node as similarItem, score

// 최종 추천 점수 계산
RETURN similarItem,
  (1.0 + score) * (COALESCE(item.popularity, 0.5)) as recommendation_score
ORDER BY recommendation_score DESC
```

5. 의미 검색 및 지식 그래프

자연어 쿼리 기반 검색:

```
// 질문을 벡터로 변환
WITH vectorize('What are the latest treatments for cancer?') as question_vector

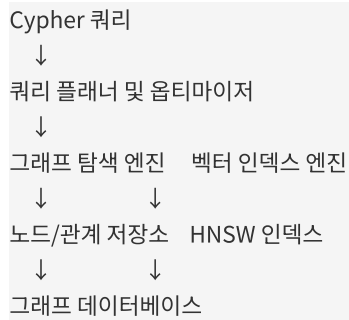
// 관련 문서 찾기
CALL db.index.vector.queryNodes('medical_doc_embedding_index', 20, question_vector)
YIELD node as document, score

// 관련 개념(노드) 탐색
MATCH (document)-[:MENTIONS]->(concept:Concept)
MATCH (concept)-[r:RELATED_TO]->(relatedConcept:Concept)

RETURN document.title, concept.name, relatedConcept.name, r.relationship_type
ORDER BY score DESC
LIMIT 10
```

33.4 아키텍처 및 성능

저장소 통합



성능 특성

검색 성능:

- 벡터 검색만: $O(\log n)$ HNSW 탐색
- 그래프 탐색만: 관계 수에 따라 선형
- 결합: 벡터 검색 후 그래프 필터링으로 최적화 가능

메모리 사용:

- 그래프 노드/관계: 구조에 따라 가변
- 벡터 인덱스: $O(nd)$ (n =벡터 수, d =차원)
- HNSW 메타데이터: $O(nm)$ (m =평균 연결 수)

확장성:

- 그래프 크기: 수십억 노드 가능 (엔터프라이즈 라이선스)
- 벡터 인덱스: 수억 벡터 가능
- 쿼리 지연시간: 수십 ms (적절한 인덱싱)

33.5 사용 사례

지식 그래프:

- 엔티티 간 의미적 연결 발견
- 오류 정정 및 중복 제거
- 질의응답 시스템

추천 시스템:

- 사용자 선호도와 아이템 유사도 결합
- 콘텐츠 및 협업 필터링 통합
- 동적 추천 재계산

검색 엔진:

- 의미적 검색
- 페이지 랭크와 벡터 유사도 결합
- 관련도 높은 페이지 발견

사기 탐지:

- 거래 패턴의 벡터 표현
- 의심 거래 네트워크 탐지
- 관계 기반 위험도 판단

33.6 본 논문과의 관계

Exqutor 은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 검색 시스템이다. Neo4j 의 벡터 검색은 다음의 측면에서 Exqutor 과 관련:

1. **하이브리드 검색:** Exqutor 의 텍스트 필터와 벡터 검색의 결합은 Neo4j 의 속성 필터와 벡터 검색 결합과 유사
 2. **관계 기반 순위 지정:** Neo4j 의 그래프 관계를 활용한 순위 지정은 Exqutor 이 검색 결과를 순위 지정할 때 고려할 수 있는 추가 신호
 3. **통합 플랫폼:** Neo4j 처럼 관계형 정보와 벡터 정보를 하나의 시스템에서 통합 관리하는 아이디어
 4. **의미 검색:** Exqutor 과 마찬가지로 자연어 쿼리의 의미를 벡터 검색으로 해석
-

추가 제기 문제

1. **그래프 + 벡터 인덱싱의 오버헤드:** 그래프 관계와 HNSW 벡터 인덱스를 동시에 유지할 때 메모리와 업데이트 오버헤드는?
2. **복합 쿼리의 최적화:** 벡터 검색 먼저 vs 그래프 필터링 먼저의 선택 기준은? 동적 최적화 가능한가?
3. **관계 기반 순위의 정확도:** 그래프 구조 정보를 벡터 점수와 어떻게 결합해야 최적의 결과를 얻을 수 있는가?
4. **대규모 그래프에서의 확장성:** 수십억 노드의 그래프에서 벡터 인덱싱과 쿼리 성능은 어떻게 변하는가?
5. **동적 업데이트:** 그래프 관계와 벡터가 함께 변할 때 인덱스의 일관성과 성능을 어떻게 유지할 것인가?
6. **임베딩 모델 변경:** 새 임베딩 모델로 모든 벡터를 재생성할 때의 전략은?